

This article was downloaded by: [Cornell University]

On: 02 June 2012, At: 11:18

Publisher: Taylor & Francis

Informa Ltd Registered in England and Wales Registered Number: 1072954 Registered office: Mortimer House, 37-41 Mortimer Street, London W1T 3JH, UK



Cryptologia

Publication details, including instructions for authors and subscription information:

<http://www.tandfonline.com/loi/ucry20>

ON THE DERIVATION OF A "CHAOTIC" ENCRYPTION ALGORITHM

Robert Matthews^a

^a 10, Park House, 55, Highbury Park, London N5 1TH ENGLAND.

Available online: 04 Jun 2010

To cite this article: Robert Matthews (1989): ON THE DERIVATION OF A "CHAOTIC" ENCRYPTION ALGORITHM, Cryptologia, 13:1, 29-42

To link to this article: <http://dx.doi.org/10.1080/0161-118991863745>

PLEASE SCROLL DOWN FOR ARTICLE

Full terms and conditions of use: <http://www.tandfonline.com/page/terms-and-conditions>

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden.

The publisher does not give any warranty express or implied or make any representation that the contents will be complete or accurate or up to date. The accuracy of any instructions, formulae, and drug doses should be independently verified with primary sources. The publisher shall not be liable for any loss, actions, claims, proceedings, demand, or costs or damages whatsoever or howsoever caused arising directly or indirectly in connection with or arising out of the use of this material.

ON THE DERIVATION OF A “CHAOTIC” ENCRYPTION ALGORITHM

Robert Matthews

ADDRESS: 10, Park House, 55, Highbury Park, London N5 1TH ENGLAND.

ABSTRACT: Under certain conditions, even simple non-linear iterative functions are capable of generating “chaotic” sequences of random numbers. In this paper we derive such a “chaotic” function, and show that it may be suitable for cryptographic use, in particular as a replacement for the one-time pad system.

KEYWORDS: Non-linear functions, chaos, random numbers, one-time encryption.

INTRODUCTION

Over the last decade, there has been considerable excitement over the discovery that certain apparently well-defined physical systems have the ability to exhibit unpredictably “chaotic” behaviour under certain circumstances.

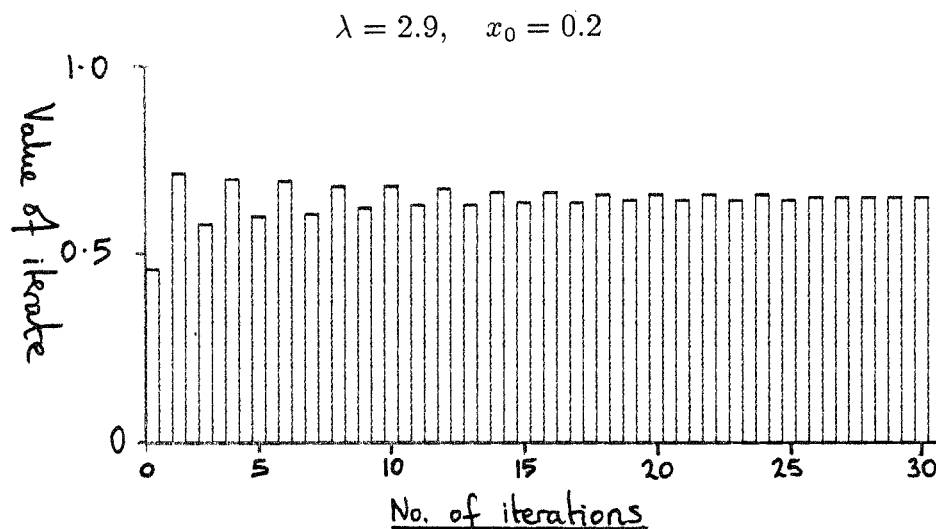
The growth of fish populations, the behaviour of electrical systems and even dripping faucets have been shown to be capable of behaving chaotically, with just small changes in the initial conditions leading to wildly different results (see, e.g. [1], [2], [3]).

The mathematics behind these systems is in some cases surprisingly simple. In particular, one-dimensional non-linear iterative functions are capable of showing chaotic behaviour. For special choices of the “tuning parameters” which control their output, it is possible to generate sequences of random numbers using such functions¹. Being chaotic functions, they also have the key characteristic, which we shall exploit here, of generating completely different sequences through relatively small changes in the tuning parameters.

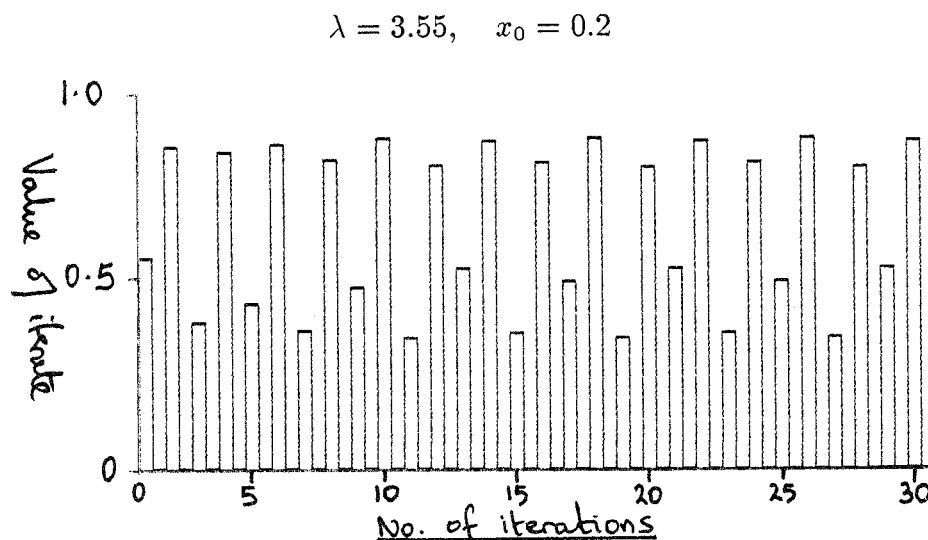
The apparent ability of such functions to provide a source of random numbers is clearly of considerable interest in cryptology. The provably unbreakable one-time system, in which plaintext is encrypted by modular addition to a key

¹Strictly, only pseudo-random numbers can be generated by a deterministic process. However, it is in principle possible to algorithmically generate a pseudo-random sequence which is sufficiently random for any reasonable cryptologic demand. We shall therefore drop the distinction between random and pseudo-randomness in what follows.

sequence of random numbers that is used only once, has never gained wide acceptance because of the difficulty of securely distributing the sequences of random numbers, the "one-time pads", to the members of a communication network.



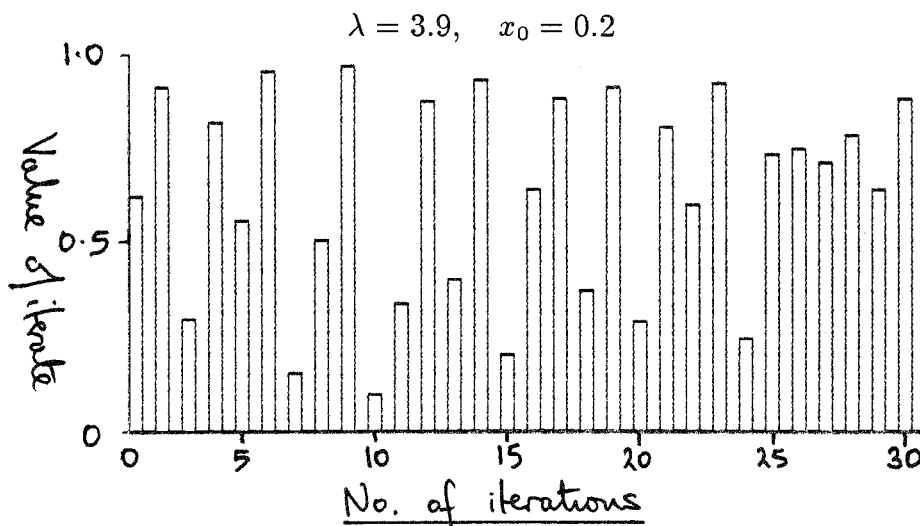
After 26 iterations, function has converged to a single value of ≈ 0.655 .



After 7 iterations, function settles down to repeating a cycle of period 8.

Figure 1. How the behaviour of $f(x_n) = \lambda x_n(1 - x_n)$ varies with λ .

If a *function* were used to generate the random keys, instead of the pads, the key sequence could be generated when and where required, using specific values of the function's tunable parameters. These parameter values would then be the only numbers that have to be disseminated, a task posing a much lower security risk than the safe distribution of one-time pads.



The function has become "chaotic", with iterates showing no signs of periodicity or convergence.

Figure 1. (Cont.) How the behaviour of $f(x_n) = \lambda x_n(1 - x_n)$ varies with λ .

Over the years a number of techniques for algorithmically generating random keystreams have been put forward, e.g. the so-called "mid-square" technique, in which a seed-number is squared and its middle digits manipulated in a certain way. Such techniques suffer from a variety of drawbacks; the mid-square technique is cumbersome, and is impractical to carry out by hand, for example.

In what follows, we shall show that chaotic functions may provide a practical solution to this long-standing problem with one-time systems. We shall briefly describe the behaviour of chaotic functions, and then derive the form of such a function which could be used in cryptologic applications.

It will be seen that the function has sufficient sensitivity to initial conditions to allow an essentially unlimited number of infinitely long random keystreams

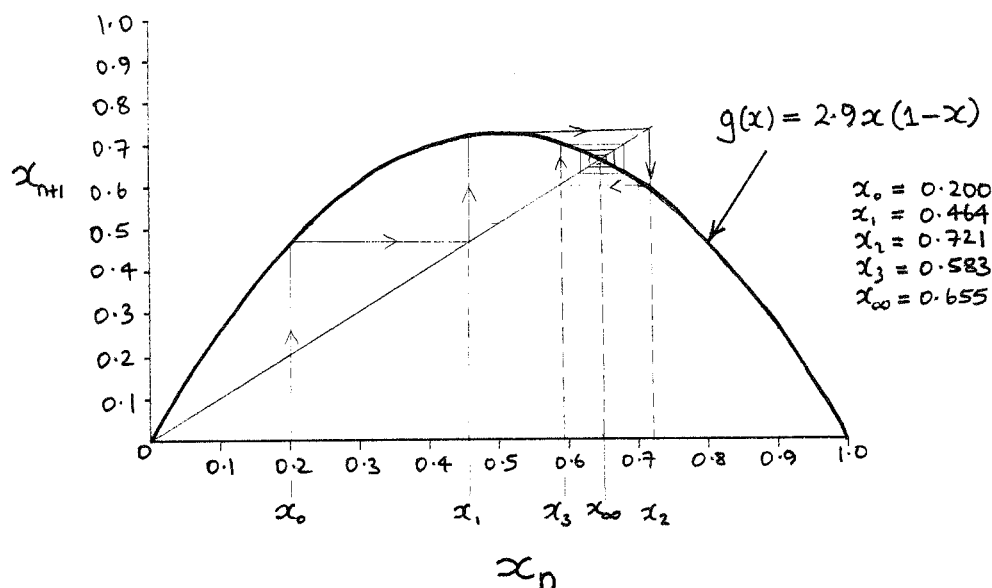


Figure 2. Iterates converge to a single value.

to be generated. Even so, its simplicity will enable even a relatively unsophisticated pocket calculator to be turned into a sophisticated cipher machine.

THE CHAOTIC BEHAVIOUR OF NON-LINEAR FUNCTIONS

The function we will derive belongs to the class of one-dimensional non-linear functions whose properties have attracted considerable interest in the last decade or so. One of the best-studied such functions (see, e.g. [1]) is the iterative mapping:

$$g(x_n) = x_{n+1} = \lambda x_n(1 - x_n) \quad (0 < x_n < 1) \quad (1)$$

where the initial value of $x(\equiv x_0)$ and λ are tunable parameters. This equation has been used to model biological populations (see, e.g. [3]). Yet despite its apparent simplicity, it is capable of producing chaotic behaviour as λ is changed, as we shall now see.

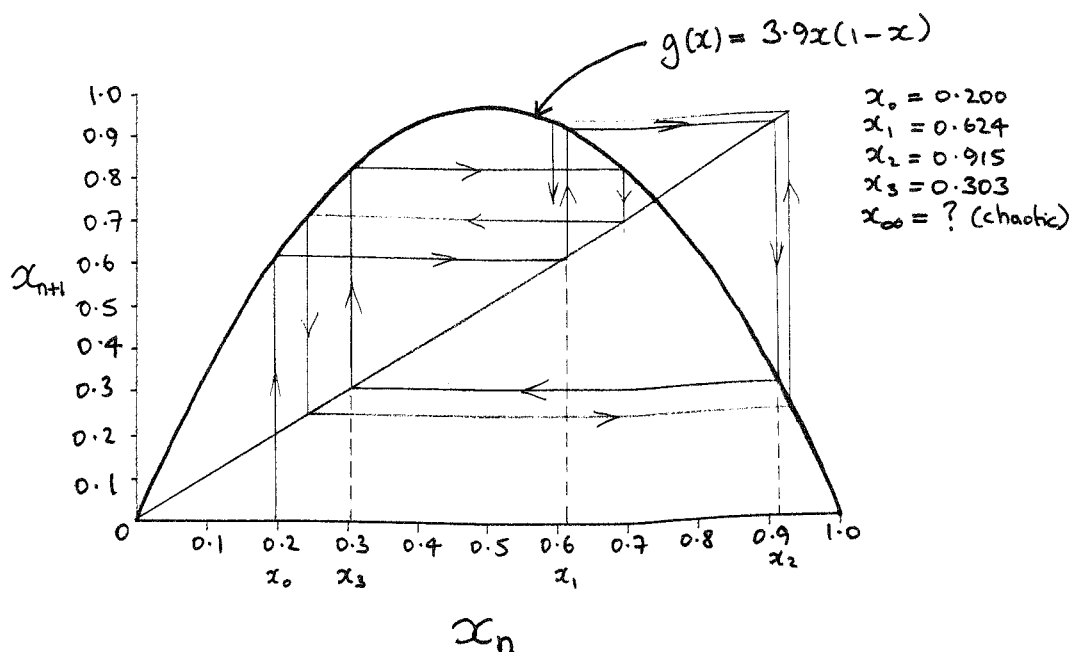


Figure 3. Iterates never converge: $g(x)$ is generating "chaos".

For values of λ no greater than 3, nothing very exciting happens: the successive values of the iterates x_n converge to a single value. However, as λ is increased above 3, the sequence starts to show periodicity. First, the sequence oscillates between two values, then between four values, then eight and so on as λ is gradually increased. However, at some values of λ above about 3.6, the iterates appear to become completely random. Above about $\lambda = 3.8$ the "period doubling" has vanished completely, and every sequence of iterates generated by a given x_0 is random: the function has become completely "chaotic". See Figure 1.

Figures 2 and 3 plot $g(x)$ in the two cases of $\lambda = 2.9$ and $\lambda = 3.9$. The line cutting across the graphs is the locus of points for which $g(x) = x$. If we choose a starting iterate x_0 along the x-axis, the next iterate is given by moving vertically upwards to the curve of $g(x)$, then moving horizontally to the line of $g(x) = x$. Vertically below this point is the value of the second iterate; to generate the third, then, we simply move vertically upward from the $g(x) = x$ line to intersect the curve again, and repeat the process.

Considerable effort has been expended over the last decade in developing

a mathematical understanding of chaotic behaviour in functions such as $g(x)$. The papers of Feigenbaum ([4, 5]), which laid the foundation for much of the current work in chaos theory, deal with this in detail.

A key concept in the understanding of chaotic behaviour of a given function is that of the *fixed points* of the function. These are the values of x ($= x_n$; we shall drop subscripts henceforth for simplicity) for which the function simply produces x again. In equation (1), the value $x = 0$ is a trivial fixed point (hence the condition on x). The other fixed point is given by solving the equation

$$x' = \lambda x'(1 - x')$$

From which we find

$$x' = 1 - 1/\lambda. \quad (2)$$

In Figures 2 and 3, the fixed points are where the line of $g(x) = x$ intersects with the curve of $g(x)$.

For certain values of λ , the fixed points of a function become unstable; that is, no matter how close the initial value of x is made to the fixed point, subsequent iterates diverge away from it. It is this instability of the fixed points of (1) that accounts for the loss of convergence to a single value for certain values of the parameter λ . As Feigenbaum shows ([4]), a fixed point is unstable if the modulus of the slope of the function at the fixed point is greater than 1.

We can now see the significance of the values of λ given in the qualitative description of the behaviour of equation (1) given earlier. For equation (1), $dg(x)/dx = \lambda(1 - 2x)$. When $0 < \lambda < 1$, the fixed point $x' = 0$ is stable, according to Feigenbaum’s rule. Substituting (2) to investigate the other fixed point we get $dg(x')/dx = 2 - \lambda$, which shows that there are no stable fixed points at all if $\lambda > 3$.

Above $\lambda = 3$, then, the function will not converge, and period doubling and chaos will start to appear. However, above this value of λ , the analytical description of the behaviour of equation (1) becomes extremely complex (see e.g. [5]).

Feigenbaum has, however, derived an approximate technique for calculating the value of λ , denoted by λ_∞ , above which at least some of the period doubling becomes of infinite length; in other words, where aperiodic “bursts” of chaos can occur.

This is, however, not good enough for our cryptographic purposes. Even above λ_∞ , there are so-called “windows of stability” between the bursts of

chaos. Being periodic, the sequence of iterates generated in such windows offers virtually no security as a keystream.

So what are the requirements for tuning parameters of a generalised chaotic function that make it suitable for cryptologic application?

THE GENERALISED CHAOTIC FUNCTION

Thus far we have considered the simple 1-dimensional iterative mapping given by equation (1). This has two variables, λ and the initial value for x . Consider now the generalised version of that equation:

$$g(\lambda, \alpha, \beta, x) = \lambda x(\alpha - x)^\beta. \quad (3)$$

Here we have, apparently, four variable parameters, which, because of the sensitivity to initial conditions of such function, are capable of producing a very large number of different sequences of random numbers.

We shall now derive the conditions on the tuning parameters that have to be satisfied to generate random number sequences. This will show that, in fact, λ is not an independent tunable parameter.

First, the function must possess a differentiable z^{th} order maximum, such that $g(0) - g(x) \propto |x|^z, z \geq 1$ for small $|x|$ (see [5], page 209). The simplest way of ensuring this is to have λ, α and β all positive.

Next, the function must generate non-periodic sequences of iterates. Feigenbaum's work shows that the crucial feature of a function such as (3), which determines whether or not it behaves chaotically, is the *value of gradient of $g(x)$ at its non-trivial fixed point*. As we have already seen, if the modulus of the gradient is less than unity, the iterates will converge to a single value.

Feigenbaum went on to show that the value of λ above which at least bursts of chaos appear, i.e. λ_∞ , could be calculated from the condition that the gradient of $g(x)$ at its non-trivial fixed point be greater than -1.6012.

The general principle is that the bigger the modulus of the gradient at the fixed point, the more bizarre the behaviour of the function. We now show that combining this rule with a number of constraints set by practical considerations enables us to find the cryptologically-useful range of the tunable parameters.

We have

$$g(x) = \lambda x(\alpha - x)^\beta.$$

Hence

$$dg(x)/dx = \lambda(\alpha - x)^\beta - \lambda x\beta(\alpha - x)^{\beta-1}. \quad (4)$$

For $g(x)$ we find that a non-trivial fixed point $g(x') = x'$ exists at

$$x' = (\alpha - 1/\lambda^{1/\beta}).$$

Hence the gradient at the fixed point is given by

$$dg(x')/dx = 1 + \beta - \alpha\beta\lambda^{1/\beta}. \quad (5)$$

It is the right hand side of (5) that we must maximise to force the function into chaotic behaviour. But how we maximise it depends on the location of the non-trivial fixed point. If it lies to the right of the central hump of the function (that is, if x' is greater than the value of x for which $dg(x)/dx = 0$), then we need to make $dg(x')/dx$ as *negative* as possible.

Now $dg(x)/dx = 0$ if $x = \alpha/(1 + \beta)$, giving a maximum value of $g(x)$, denoted by g_m , of

$$g_m = \lambda[\alpha^{\beta+1}/(\beta + 1)](1 + 1/\beta)^{-\beta}.$$

Rearranging, we find

$$\lambda = (\beta + 1)[g_m/\alpha^{\beta+1}](1 + 1/\beta)^\beta. \quad (6)$$

Now, the fixed point lies to the right of the hump if

$$\alpha - 1/\lambda^{1/\beta} > \alpha/(1 + \beta).$$

Rearranging and substituting for λ from (6), we find that the criterion for the fixed point lying to the right of the hump is that

$$g_m > \alpha/(1 + \beta). \quad (7)$$

Now, as (3) shows, for real values of $g(x)$, α must be at least as great as the maximum value of any of its iterates (that is, g_m), i.e. $\alpha = kg_m$, $k \geq 1$. Substituting this in (7), we find that the fixed point will be to the right of the central hump if $\beta > 0$. So, for our purposes, we must make the gradient at the fixed point as *negative* as possible.

Writing $dg(x')/dx$ as $(1 + \beta) - (g_m/\alpha)^{1/\beta}(1 + \beta)^{1+1/\beta}$, we see that this is achieved by setting $g_m \gg \alpha$. But, for real values of $g(x)$, we must have $\alpha \geq g_m$, so the optimal case is where

$$\alpha = g_m. \quad (8)$$

Substituting this in (6), and substituting for λ in (3) we see that the form of $g(x)$ which has the maximum gradient at its fixed point is given by

$$g(x) = [(\beta + 1)/g_m^\beta](1 + 1/\beta)^\beta x(g_m - x)^\beta. \quad (9)$$

Cryptologically, an upper limit must be put on g_m . Without this, there is a danger that $g(x)$ could become very small when β is large, with the result that a sequence of zeroes will crop up in the key-stream generated by the function. Such iterates offer no security in encryption. This risk is most simply avoided by setting $g_m = 1$. By (8), this implies $\alpha = 1$ also, and so we now arrive at the cryptologically-useful form of (3):

$$g(x) = (\beta + 1)(1 + 1/\beta)^\beta x(1 - x)^\beta. \quad (10)$$

We see that λ and α are not, in fact, tunable parameters, and that the initial value of x must lie between 0 and 1.

What are the constraints of β ? These can be set by finding the value for the gradient of $g(x)$ at its fixed point at which chaos occurs. From Feigenbaum's approximation technique mentioned earlier, we know that this value must exceed -1.6012, as this is the value of the gradient at which at least bursts of chaos appear in the iterates.

We can improve on this, and set a lower bound on β , by exploiting an investigation by Ulam and Von Neumann [6], carried out before the term chaotic functions had been coined, into the properties of a special case of our generalised function. They found that the iterative formula $g(x) = 4x(1 - x)$ was capable of producing a series of iterates whose statistical distribution implied that $g(x)$ suitable for use as a random number generator. Using the formulae derived above, it can be shown that the value of the gradient at this function's fixed point is -2.

Inserting this into the equation for $dg(x')/dx$ shows that the lower limit on β set by the need for $g(x)$ to have a gradient of at least -2 at its fixed point is $\beta \geq 1$.

An upper limit on β is set by computational considerations. Suppose the calculator or computer being used to calculate the iterates has a D -digit display, and a computational range of $10^{-n} \leq x \leq 10^{+n}$. Then if an iterate x in the sequence is so close to unity that $(1 - x)^\beta \leq 10^{-n}$, the calculator/computer will throw up an error message such as "Out of range". To cure this, we set a limit on β such that

$$(10^{-D})^\beta > 10^{-n}.$$

This puts an upper limit on β of n/D . On the computer used by the author (an Amstrad PCW 8256), n is 38 and D is 8 on single precision; hence we can

use

$$g(x) = (\beta + 1)(1 + 1/\beta)^\beta x(1 - x)^\beta$$

with $1 \leq \beta \leq 4.$

as reasonable bounds on β .

We have now arrived at the form of our chaotic encryption algorithm, and the bounds of its operation.

USING THE CHAOTIC FUNCTION IN CRYPTOGRAPHY

We now describe how the generalised chaotic function can be used in practical cryptography, using relatively unsophisticated equipment. The range of $g(x)$ is such that its output will consist of a series of numbers less than unity, specified to D places of decimals, where D is set by the design of calculator or computer being used. The series will be generated by $g(x)$ using specified values of β and x_0 , the initial iterate.

By taking the last two digits (i.e. those two closest to the *rhs*) of the iterates generated, and converting them to a convenient base, the result can be added to a plaintext to give a ciphertext consisting of letters that have been encrypted using a random keystream. To decrypt the ciphertext, it is only necessary to plug the same values of β and x_0 into $g(x)$, and subtract the resulting iterates from the ciphertext.

The encryption can conveniently be done by hand using a calculator and a device described by the author in an earlier paper [7] that is capable of converting numbers between 0 and 99 into modulo 25.

First, we decide on the value of β we are going to use to encrypt our plaintext. Although this must lie within the appropriate limits as explained above, it is important to realise β does *not* have to take an integer value. Indeed, the beauty of the chaotic function is that it will give entirely different keystreams for relatively close non-integer values of β . Numerical simulations show that suitably different keystreams are produced if β is specified to about two significant figures fewer than the iterates. So, if the iterates are calculated to $D = 10$ digits, β should be specified to $D - 2 = 8$ digits.

Suppose we choose $\beta = 2.53$; $g(x)$ is then $8.198790355x(1 - x)^{2.53}$. Next, we choose the starting value of the iteration, x_0 ; suppose this is 0.45. The first

five iterates are then 0.812980077, 0.095875139, 0.609135158, 0.463757308, and 0.785823223. The exact values may vary slightly from calculator to calculator, because of the rounding errors, etc. It is therefore important that each member of a communication network uses the same model.

The last two digits of these numbers are then taken and, using the rotor device, converted modulo 25 to give: 2, 14, 8, 8 and 23. The rotor device can then be used to add them to the plaintext letters, e.g. CHAOS, to give ciphertext, in this case EVIWQ.

To give some indication of the function's sensitivity to initial conditions, we note that if we repeat the above procedure, this time using $\beta = 2.5300001$, we find that the key sequence becomes 23, 1, 3, 12, 23.

Decryption requires the generation of the sequence of iterates, made possible only by exact knowledge of β and x_0 , and subtraction of the modulo-reduced numbers generated.

Before ending this section on the practical use of the algorithm, we should point out that there is another situation in which underflow, and hence potentially dangerous sequences of zeros, can be generated by $g(x)$. This is when the function produces an iterate x which is smaller than 10^{-D} .

The way to combat this is to convert the offending iterate into its logarithm, and multiply its modulus by a small number, say 0.001. For example, suppose $D = 8$, and the iterate happens to be precisely 10^{-10} . This iterate, and maybe its next few successors, would all give two-digit key-numbers of 00. However, by switching to logarithms, the iterate becomes -10. Taking its modulus, and multiplying by 0.001 we get 0.01; this is the iterate we use to continue the sequence.

This appears to be a complicated procedure. However, it turns out to be needed only when β is near its maximum permissible value of 4. It can easily be put into effect if the algorithm is being executed on a computer: a conditional goto loop which carries out the conversion when triggered by an iterate which is smaller than 10^{-D} is easily built into the program.

If a calculator is being used, all that needs to be done is to watch out for the occasional values of iterates which are *greater than* $1 - 10^{-D/\beta}$, as the next iterate will then fall below the limit given above, and will need conversion in the prescribed manner. For example, if the calculator has a 10-digit display, any iterate above 0.99 is likely to be followed by an iterate needing conversion. The exact value at which the logarithmic conversion is brought in must be agreed between the members of the communication network.

ESTIMATE OF SECURITY

An upper estimate of the security of the chaotic function method described here is given by the number of keys that would have to be tried in an exhaustive key search attack on an unknown text.

The function is sensitively dependent on the values of the parameters β and x_0 . Hence the number of key sequences is given approximately by the product of the number of different values the parameters can take which give rise to different key sequences. As already noted, radically different sequences are generated when the two parameters β and x_0 are specified to about two significant figures fewer than the output. So if we have a calculator/computer capable of displaying D digits, then the number of keys will be

$$K = 10^{2D-4}.$$

So, for a calculator capable of displaying 10 digits, the algorithm will generate about 10^{16} keys.

Under the usual assumption of a random cipher model (all keys equally likely) the key entropy (see e.g. [8]) satisfies

$$2^{H(K)} = 10^{(2D-4)}.$$

Converting to base-10 logarithms, we then find

$$H(K) = 6.6(D - 2).$$

This means that the complexity function of the chaotic cipher system is exponential with time (see [8], page 30), and so brute force attack against the cipher using the best computer technology of the day can always be foiled by simply increasing the number of digits, D , to which $g(x)$ is specified to an appropriate level.

Suppose a purpose-built computer is capable of R operations a second. The time, in years, needed to break the cipher by brute force will then be at least Y , where

$$Y = 2^{6.6(D-2)} / 3.10^7 R \text{ years.}$$

According to this, a brute force attack by a machine comprising 1,000 T-800 Inmos Transputers each capable of $2 \cdot 10^6$ flops will take about 1,000 years to break a chaotic cipher system with a 12-digit display. Y is, moreover, very sensitively dependent on D , with every extra digit boosting Y 100-fold. Thus if $D = 16$, the cipher would take longer than the age of the universe to break.

Despite this impressive result, it must be pointed out that brute force may not be the only way of breaking the chaotic system. It is well known that ciphers based on analytical random number generators exploiting linear modular congruences can be broken by analysing the sequence of iterates and deriving the tunable parameters of the generator. The non-linear nature of the function $g(x)$ derived here should give it much greater security against such cryptanalysis. However, a rigorous demonstration, or otherwise, of the “trapdoor” nature of $g(x)$ is required to test this conjecture.

CONCLUSION

We have derived a “chaotic” encryption algorithm $g(x)$ which generates a large set of random numbers in a repeatable fashion, which is, at least in principle, suitable for use as a replacement for one-time pads.

No sophisticated equipment is required to use the algorithm. A calculator with at least four memories, or a programmable calculator, can be used to generate the key-stream, which is then converted modulo 25 and added to the plaintext using the rotor device described in reference [7].

By enabling random number sequences to be generated when and where necessary, the function overcomes the need to securely distribute the sequences throughout a communication network, the central practical weakness of the one-time concept as currently used.

REFERENCES

1. Cvitanovic, P. 1984. *Universality in Chaos*. Bristol: Adam Hilger.
2. Moon, F. C. 1987. *Chaotic Vibrations*. New York: John Wiley.
3. Gleick, J. 1987. *Chaos: Making a New Science*. New York: Viking.
4. Feigenbaum, M. J. 1980. Universal behavior in non-linear systems. *Los Alamos Science*. 1: 4-27.
5. Feigenbaum, M. J. 1979. The universal metric properties of non-linear transformation. *Journal of Statistical Physics*. 21: 669-706.
6. Ulam, S. M. and J. von Neumann. 1947. On combinations of stochastic and deterministic processes. *Bull. Am. Math. Soc.* 53: 1120.
7. Matthews, R. A. J. 1988. A rotor device for periodic and random-key encryption. *Cryptologia*. (In the press).
8. Denning, D. E. R. 1983. *Cryptography and Data Security*. Reading MA: Addison-Wesley.

BIOGRAPHICAL SKETCH

After graduating in physics from the University of Oxford in 1981, Robert Matthews became a science journalist, and is currently Technology Correspondent of *The Times* in London.

CRYPTOLOGIA ANNUAL UNDERGRADUATE PAPER COMPETITION IN CRYPTOLOGY

FIRST PRIZE: Three hundred dollars

Closing date: 1 January

Topic may be in any area of cryptology
technical, historical, and literary subjects.

Announcing a contest to encourage the study of all
aspects of cryptology in the undergraduate curricula.

Papers must be no more than twenty typewritten pages in length, double spaced and fully referenced. Four copies must be submitted. Authors should retain one copy. Papers are to be original works which have not been published previously.

The papers will be judged by the editors of *Cryptologia* and the winner will be announced on 1 April with publication of the winning paper in a future issue of *Cryptologia*.

The competition is underwritten by a generous gift from Boshra H. Makar,
Professor of Mathematics, Saint Peter's College, Jersey City NJ.

Cryptologia is a unique scholarly journal devoted to all aspects of cryptology. The journal began quarterly publication in 1977. Areas covered include computer security, history, codes and ciphers, mathematics, military science, espionage, cipher devices, literature, and ancient languages. Features of the journal include reviews of literature and equipment, news of the crypto community, announcements of activities, challenging ciphers, and exchange column.

Inquiries, submissions, and subscription information:

Cryptologia, Editorial Office
Rose-Hulman Institute of Technology
Terre Haute IN 47803 USA.